Signhify Hunter — Backend PRD-TRD

> **Codename:** HUNTER · **Version:** 1.0 · **Date:** 07 Aug 2026 Scope: API, agents, queues, delivery engine, compliance, billing. Companions: `02_TRD.md` , `03_IMPLEMENTATION_PLAN.md`

---

1. Product Requirements (Backend)

1.1 Goals

1. Continuous, autonomous lead pipeline: source → enrich → verify → qualify → write → deliver → respond — with zero silent failures.
2. At-least-once everything, idempotent everything; replays are safe.
3. Delivery that protects sender reputation (warmup, limits, suppression, alarms).
4. Compliance first: unsubscribes instant, suppression permanent, DSAR workable, retention enforced.
5. Every AI decision auditable (event log with model, tokens, latency, input hash).

1.2 System-level non-negotiables

#	Requirement
B-1	All queue handlers idempotent (natural-key upsert; `provider_message_id` unique)
B-2	No PII in logs; structured logs only; OpenTelemetry traces per lead-id
B-3	RLS on every table; service-role key used only in worker processes, never in browser
B-4	Send-time suppression check mandatory (cache + DB), even in sandbox
B-5	Global rate caps enforced at queue layer AND at provider layer (defense in depth)
B-6	Any agent failure → job retried 3× (backoff) → DLQ → alert; never silently dropped
B-7	Budget caps per campaign/workspace with hard stop + alarm

---

2. API Design (routes grouped)

2.1 Auth (Supabase)

- Session-based; middleware validates JWT → workspace id; all queries scoped.

2.2 Leads

- `GET /api/leads` — filters: `source, tier, verification, status, channel, q, page, pageSize`; sortable.
- `GET /api/leads/:id` — detail + enrichment + events + messages.
- `POST /api/leads/:id/tier` — manual tier override (audit event).
- `POST /api/leads/:id/reverify` — re-run verify pipeline.
- `POST /api/leads/bulk-tier` — bulk override.
- `GET /api/leads/export` — CSV (respects 50k cap, async job + download URL).

2.3 Sources

- `GET/POST/PATCH/DELETE /api/sources`
- `POST /api/sources/:id/run` — enqueue immediate scout run.
- `GET /api/sources/:id/status` — last run, counts, errors.

2.4 Campaigns

- `GET/POST /api/campaigns`; `GET/PATCH/DELETE /api/campaigns/:id`
- `POST /api/campaigns/:id/run` (validate → count audience → enqueue), `POST /api/campaigns/:id/pause`, `POST /api/campaigns/:id/resume`
- `POST /api/campaigns/:id/preview-samples` — generate 5 AI samples (LLM, cost-capped).
- `GET /api/campaigns/:id/analytics` — step stats, funnel, budget.

2.5 Inbox

- `GET /api/inbox?status=&category=` ; `GET /api/inbox/:threadId`
- `POST /api/inbox/:threadId/reply` — `{body}` → validates approved-by → sends (idempotent via thread+provider key).
- `POST /api/inbox/:threadId/book` — create Cal.com event + lead → `meeting`.
- `POST /api/inbox/:threadId/regenerate` — new AI suggestion.

2.6 Analytics

- `GET /api/analytics/dashboard?range=` — KPIs + funnel + deliverability.
- `GET /api/analytics/channels` ; `GET /api/analytics/export`

2.7 Settings

- `GET/POST /api/settings/domains` + `GET /api/settings/domains/:id/check` (DNS verification)
- `GET/PATCH /api/settings/icp` — ICP rules (qualify agent reads).
- `GET/POST /api/settings/integrations` — booking link, CRM webhook, Stripe portal link.
- `GET /api/usage` — metered counts.

2.8 Compliance (public)

- `GET /api/compliance/unsubscribe?email=&token=` — validates token (HMAC w/ 30-day expiry) → suppression insert + lead `unsubscribed`.
- `POST /api/compliance/dsar` — `{email, action: export|erase}` → queued job → email result.

2.9 Webhooks

- `POST /api/webhooks/resend` — inbound email (message → thread), events (delivered/opened/clicked/bounced/complained). Signature-verified.
- `POST /api/webhooks/stripe` — subscription + usage billing events.

---

3. Queue Architecture

```
Upstash Redis streams per stage (scout, enrich, verify, qualify, write, deliver, warmup, reply,
Workers: Node processes (local: `pnpm worker`; prod: Cloudflare Workers / Vercel cron + queue t
```

- Job payload: `{jobId, workspaceId, payload, attempt, idempotencyKey}`.
- Handler pattern (every worker):

```
export const handler = async (job: Job) => {
  const key = `done:${job.idempotencyKey}`;
  if (await redis.exists(key)) return;            // already processed
  const result = await process(job.payload);      // business logic
  await redis.set(key, "1", { ex: 60 * 60 * 24 });// mark done
  await logEvent(job);                            // audit trail
  return result;
};
```

- Retries: 3 attempts, exponential backoff (1s/10s/60s), then DLQ stream + alert.
- Cron jobs (Vercel cron): scheduler every 5 min (due sends), warmup daily 09:00 UTC, retention daily 02:00 UTC, re-score daily for Tier B (14d cadence), DNS/blacklist check daily.

---

4. Delivery Engine Spec

4.1 Send flow

1. Scheduler picks `CampaignLead` where `next_send_at <= now` (limit batch 200).
2. Checks (all must pass): suppression, daily domain cap, global cap, warmup level, send window, not-replied.

3. Compose via Resend with headers: `List-Unsubscribe: <https://signhify.dpdns.org/compliance/unsubscribe?…>` + `List-Unsubscribe-Post: List-Unsubscribe=One-Click`; physical address block; truthful subject.
4. Mark `sent` with `provider_message_id`; event log.
5. On bounce: 550/hard → suppression + lead status `unsubscribed`; soft → retry next window (max 3).
6. On reply/unsubscribe/complaint: sequence stops for that lead immediately; complaint → alert + domain score check.

4.2 Warmup engine

- 14-day ramp: day n sends = `min(5 * ceil(n/2), 30)` to warmup network (Seedleads API or self-hosted pool of test inboxes).
- Verifies inbox placement via seeded test addresses; stops + alerts if inbox rate < 90%.
- Domain state machine: `provisioning → warming → active → cooling`.

4.3 Limits (defaults, configurable)

Knob	Default	Hard cap
Per-domain daily	30 → 150 (after warmup)	150
Per-campaign daily	50	workspace cap
Per-lead touches	3 steps / campaign	3 campaigns ever
Min step delay	2 days	—
Send window	9:00–17:00 local lead tz	—
Complaint alarm	0.1% of last 500	auto-stop campaign

---

5. Agent Workers (backend contract)

Worker	Input	Output	Model	Notes
`scout`	`{sourceId, cursor}`	`Lead[]` (upsert by domain)	none (adapters)	per-adapter rate limit; 429 → backoff; ToS-safe endpoints only
`enrich`	`{leadId}`	`Enrichment`	none (APIs/heuristics)	Hunter.io/Apollo optional, budget-capped
`verify`	`{leadId, email}`	`EmailVerdict`	none (SMTP probe)	3s timeout; catch-all → `risky`

Worker	Input	Output	Model	Notes
`qualify`	`{leadId, icpRules}`	`{score, tier, reasons[]}`	gpt-4o-mini	deterministic pre-filter in SQL first
`writer`	`{leadId, campaignId, step}`	`{subject, body}`	Claude Sonnet 4	120-word cap; honesty rules; fallback = template fill
`ops.send`	`{messageId}`	`{status}`	none	all §4 gates
`ops.warmup`	`{domainId}`	`{count, inboxRate}`	none	daily
`respond.classify`	`{threadId}`	`{category, intent, suggestion, needsHuman}`	gpt-4o-mini	unsubscribe auto-handled, no LLM
`respond.book`	`{threadId}`	`{bookingUrl}`	none	Cal.com API

6. Compliance Implementation

Mechanism	Implementation
One-click unsubscribe	HMAC token (workspace secret, 30d) → public route → suppression row + `Message.unsubscribed` + lead stop
`list-unsubscribe`	Added to every message + POST header
Physical address	Signhify registered address block, footer of every email
GDPR/DPDP	Lawful basis noted per lead (legitimate interest); DSAR export/erase jobs; retention 90d; suppression permanent & re-checked on future ingest
Data residency	Supabase region chosen at setup (EU if EU prospects targeted)
Audit	`Event` table: approvals, sends, tier changes, DSAR actions, by whom
ToS guardrails	LinkedIn/X/Reddit/Upwork adapters: public API/feed only; DM suggestions are copy-ready, human-send (no automation)

7. Observability & Ops

- Health: `GET /api/health` (db, redis, resend, queue depths).

- Metrics (Axiom): jobs by stage (count, latency, error rate), send/bounce/reply rates, LLM cost/day, warmup inbox rate, queue depth alerts.
- Alerts (founder): complaint > 0.1%, bounce > 5%, blacklist hit, queue DLQ non-empty, LLM budget > 80%, delivery inbox < 95%.
- Dashboard: real-time agent activity via Realtime + `Event` table.

---

8. Acceptance Criteria (backend)

1. Scout: 2 sources × 100 leads, zero dupes on re-run, all jobs idempotent (replay test passes).
2. Verify: 100-lead batch → ≥ 85% verified; verdicts deterministic on replay.
3. Delivery: sandbox send → events flow; hard bounce auto-suppresses; reply auto-stops sequence (tested end-to-end).
4. Limits: scheduler never exceeds caps even under 10× load (load test with 5k due sends).
5. Compliance: unsubscribe < 5s to suppression; DSAR export/erase runs; retention job deletes 90d-old untouched leads.
6. Observability: alert fires on simulated complaint spike; DLQ replay works from UI.